RetinAI

Fondements Mathématiques et Algorithmiques du Pipeline

Analyse détaillée des algorithmes de Computer Vision appliqués à la détection de rétinopathie diabétique

Pipeline ML · TensorFlow · Computer Vision

Mars 2026

Document pédagogique — Toutes les notions sont expliquées de zéro.

Table des matières

1 Introduction **2**

2 Prétraitement des images rétiniennes **2**

2.1 Espace colorimétrique et représentation des images . . . 2

2.2 Normalisation Ben-Graham . . . 2

2.2.1 Filtre gaussien . . . 2

2.2.2 Formule de normalisation . . . 3

2.3 CLAHE — Contrast Limited Adaptive Histogram Equalization . . . 3

2.3.1 Égalisation d'histogramme classique . . . 3

2.3.2 Adaptive Histogram Equalization (AHE) . . . 3

2.3.3 Limitation du contraste (CLAHE) . . . 3

2.4 Extraction de la ROI circulaire . . . 4

3 Réseaux de neurones convolutifs **4**

3.1 Convolution 2D . . . 4

3.2 Fonctions d'activation . . . 5

3.3 Batch Normalization . . . 5

3.4 EfficientNetV2 . . . 5

3.4.1 Scaling composé . . . 5

3.4.2 MBConv et Fused-MBConv . . . 5

3.4.3 Squeeze-and-Excitation (SE) . . . 6

3.5 Module d'attention spatiale (CBAM) . . . 6

3.5.1 Attention par canal . . . 6

3.5.2 Attention spatiale . . . 6

3.6 Transfer Learning et Fine-Tuning . . . 7

4 Fonctions de perte **7**

4.1 Cross-Entropy catégorielle . . . 7

4.2 Focal Loss . . . 7

4.3 Label Smoothing . . . 8

4.4 Quadratic Weighted Kappa (QWK) Loss . . . 8

5 Augmentation des données **9**

5.1 Augmentations géométriques . . . 9

5.2 MixUp . . . 9

5.3 CutMix . . . 9

6 Optimisation de l'entraînement **10**

6.1 Optimiseur AdamW . . . 10

6.2 Réduction du learning rate sur plateau . . . 10

6.3 Early Stopping . . . 10

7 Optimisation du modèle à l'inférence **10**
7.1 Pruning structurel 10
7.2 Quantification 11
7.3 Test-Time Augmentation (TTA) 11

8 Évaluation et métriques **11**
8.1 Matrice de confusion 11
8.2 Métriques par classe 11
8.3 AUC-ROC (Area Under the ROC Curve) 12
8.4 Expected Calibration Error (ECE) 12

9 Explicabilité : Grad-CAM **12**
9.1 Formulation 12

10 Récapitulatif des notations **14**

1 Introduction

Ce document présente de manière rigoureuse et pédagogique l'ensemble des algorithmes et fondements mathématiques utilisés dans le pipeline **RetinAI**. Chaque notion est introduite progressivement, des prérequis de base aux formulations avancées, avec des illustrations et des interprétations intuitives.

Le pipeline couvre quatre grandes familles de techniques :

1. **Prétraitement d'images** : transformations pixel-level pour normaliser et rehausser les images rétiniennes.
2. **Réseaux de neurones convolutifs** : architecture profonde pour l'extraction de caractéristiques et la classification.
3. **Fonctions de perte** : objectifs d'optimisation adaptés au déséquilibre de classes.
4. **Optimisation et inférence** : techniques de compression, augmentation à l'inférence et calibration.

2 Prétraitement des images rétiniennes

2.1 Espace colorimétrique et représentation des images

Image numérique

Une image couleur est une fonction $I : \{1, \dots, H\} \times \{1, \dots, W\} \to \{0, \dots, 255\}^3$ qui associe à chaque pixel (i, j) un triplet (R, G, B) représentant les intensités rouge, verte et bleue. On la représente comme un tenseur $\mathbf{I} \in \mathbb{R}^{H \times W \times 3}$.

L'espace **LAB** (CIE $L^*a^*b^*$) sépare la luminance L (intensité lumineuse) des composantes chromatiques a (axe vert–rouge) et b (axe bleu–jaune). Cette décomposition est exploitée par le CLAHE pour modifier le contraste sans altérer les couleurs.

2.2 Normalisation Ben-Graham

Principe

L'illumination non-uniforme dans les images de fond d'œil introduit un biais spatial. La normalisation Ben-Graham supprime cette composante basse fréquence en soustrayant un flou gaussien local.

2.2.1 Filtre gaussien

Le noyau gaussien 2D de variance σ^2 est défini par :

$$G_\sigma(x, y) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right) \tag{1}$$

La convolution de l'image I avec ce noyau produit une version lissée :

$$I_{\text{blur}}(i,j) = (I * G_\sigma)(i,j) = \sum_{m,n} I(i-m,\, j-n)\, G_\sigma(m,n) \tag{2}$$

2.2.2 Formule de normalisation

La transformation Ben-Graham s'écrit :

$$\boxed{I_{\text{norm}}(i,j) = \alpha \cdot I(i,j) - \alpha \cdot I_{\text{blur}}(i,j) + \beta} \tag{3}$$

avec typiquement $\alpha = 4$ et $\beta = 128$. Le terme $\alpha(I - I_{\text{blur}})$ isole les hautes fréquences (vaisseaux, micro-anévrismes, exsudats) tandis que β recentre les valeurs.

Intuition

Imaginez que vous regardez une photo avec un éclairage inégal (plus clair à gauche qu'à droite). Le flou gaussien capture cette tendance d'éclairage. En la soustrayant, on obtient une image où seuls les *détails* sont visibles, indépendamment de l'illumination.

2.3 CLAHE — Contrast Limited Adaptive Histogram Equalization

2.3.1 Égalisation d'histogramme classique

L'histogramme d'une image en niveaux de gris est la distribution des intensités :

$$h(k) = \#\{(i,j) : I(i,j) = k\}, \quad k \in \{0, \ldots, 255\} \tag{4}$$

L'égalisation d'histogramme applique la transformation :

$$T(k) = \left\lfloor 255 \cdot \frac{\text{CDF}(k) - \text{CDF}_{\min}}{H \cdot W - \text{CDF}_{\min}} \right\rfloor \quad \text{où} \quad \text{CDF}(k) = \sum_{i=0}^{k} h(i) \tag{5}$$

2.3.2 Adaptive Histogram Equalization (AHE)

L'AHE divise l'image en tuiles (grille $M \times N$, typiquement 8×8) et applique une égalisation locale à chaque tuile. Pour éviter les artefacts aux frontières, les résultats sont interpolés bilinéairement entre tuiles voisines.

2.3.3 Limitation du contraste (CLAHE)

L'AHE peut amplifier le bruit dans les régions homogènes. Le CLAHE résout ce problème en *écrêtant* l'histogramme local :

Algorithme 1 : CLAHE

Input : Image I, grille $M \times N$, seuil de clip τ

Output : Image rehaussée I'

Diviser I en tuiles de taille $\frac{H}{M} \times \frac{W}{N}$
pour *chaque tuile* $T_{m,n}$ **faire**
  Calculer l'histogramme local $h_{m,n}$
  excès $\leftarrow \sum_k \max(0,\ h_{m,n}(k) - \tau)$
  Redistribuer l'excès uniformément : $h'_{m,n}(k) \leftarrow \min(h_{m,n}(k),\ \tau) + \frac{\text{excès}}{256}$
  Calculer $\text{CDF}_{m,n}$ à partir de $h'_{m,n}$
fin
pour *chaque pixel* (i, j) **faire**
  Identifier les 4 tuiles voisines
  $I'(i, j) \leftarrow$ interpolation bilinéaire des 4 CDF appliquées à $I(i, j)$
fin
retourner I'

Le paramètre τ (clip limit) contrôle l'amplification maximale du contraste. Une valeur typique est $\tau = 2.0$.

2.4 Extraction de la ROI circulaire

Les images de fond d'œil contiennent un disque rétinien sur fond noir. L'extraction consiste à :

1. Seuillage binaire : $B(i, j) = \mathbf{1}[I_{\text{gray}}(i, j) > t]$ avec $t = 15$.
2. Détection du contour le plus grand (disque rétinien).
3. Calcul du cercle englobant minimal $\mathcal{C}(c_x, c_y, r)$ via l'algorithme de Welzl.
4. Crop carré centré sur (c_x, c_y) de côté $2(r + m)$ où $m = 0.05 \cdot r$.
5. Application d'un masque circulaire binaire.

3 Réseaux de neurones convolutifs

3.1 Convolution 2D

Opération de convolution

Soit une entrée $\mathbf{X} \in \mathbb{R}^{H \times W \times C_{\text{in}}}$ et un filtre $\mathbf{W} \in \mathbb{R}^{k \times k \times C_{\text{in}} \times C_{\text{out}}}$. La convolution produit :

$$Y_{i,j,f} = \sum_{c=1}^{C_{\text{in}}} \sum_{m=0}^{k-1} \sum_{n=0}^{k-1} W_{m,n,c,f} \cdot X_{i+m,\, j+n,\, c} + b_f \tag{6}$$

où b_f est le biais du filtre f.

3.2 Fonctions d'activation

ReLU. $\mathrm{ReLU}(x) = \max(0, x)$. Introduit la non-linéarité tout en préservant les gradients pour $x > 0$.

Swish / SiLU. Utilisée dans EfficientNetV2 :

$$\mathrm{Swish}(x) = x \cdot \mathrm{sigmoid}(\beta x) = \frac{x}{1 + e^{-\beta x}} \tag{7}$$

avec $\beta = 1$ par défaut. Plus lisse que ReLU, elle permet un flux de gradient non nul pour $x < 0$.

Softmax. Pour la couche de sortie (classification) :

$$\mathrm{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}, \quad i = 1, \ldots, K \tag{8}$$

Les sorties forment une distribution de probabilité : $\sum_i \mathrm{softmax}(z_i) = 1$ et $\mathrm{softmax}(z_i) > 0$.

3.3 Batch Normalization

Pour chaque mini-batch $\mathcal{B} = \{x_1, \ldots, x_m\}$:

$$\mu_{\mathcal{B}} = \frac{1}{m} \sum_{i=1}^{m} x_i \qquad \sigma_{\mathcal{B}}^2 = \frac{1}{m} \sum_{i=1}^{m} (x_i - \mu_{\mathcal{B}})^2 \tag{9}$$

$$\hat{x}_i = \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \qquad y_i = \gamma\, \hat{x}_i + \beta \tag{10}$$

où γ et β sont des paramètres appris, et $\epsilon \approx 10^{-5}$ assure la stabilité numérique.

3.4 EfficientNetV2

3.4.1 Scaling composé

EfficientNet utilise un coefficient composé ϕ pour mettre à l'échelle simultanément trois dimensions :

$$\text{profondeur} : d = \alpha^{\phi}, \quad \text{largeur} : w = \beta^{\phi}, \quad \text{résolution} : r = \gamma^{\phi} \tag{11}$$

sous la contrainte $\alpha \cdot \beta^2 \cdot \gamma^2 \approx 2$ (le FLOPS double pour chaque incrément de ϕ).

3.4.2 MBConv et Fused-MBConv

Le bloc fondamental est le **Mobile Inverted Bottleneck** (MBConv) :

1. **Expansion** : Conv 1×1 de C vers $t \cdot C$ canaux (facteur d'expansion t).
2. **Depthwise convolution** : Conv $k \times k$ par canal (efficace en calcul).
3. **Squeeze-and-Excitation** : recalibrage des canaux (voir ci-dessous).
4. **Projection** : Conv 1×1 de $t \cdot C$ vers C' canaux.
5. **Connexion résiduelle** : $\mathbf{y} = \mathbf{x} + \mathrm{MBConv}(\mathbf{x})$ si $C = C'$.

EfficientNetV2 remplace les premiers blocs MBConv par des blocs **Fused-MBConv** qui fusionnent l'expansion et la depthwise convolution en une seule Conv 3×3 standard, plus rapide sur GPU.

3.4.3 Squeeze-and-Excitation (SE)

Le module SE recalibre les canaux en fonction de leur importance globale :

$$\mathbf{s} = \sigma(\mathbf{W}_2 \, \mathrm{ReLU}(\mathbf{W}_1 \, \mathrm{GAP}(\mathbf{F}))) \in \mathbb{R}^C \tag{12}$$

$$\widetilde{\mathbf{F}}_c = s_c \cdot \mathbf{F}_c \tag{13}$$

où $\mathrm{GAP}(\mathbf{F})_c = \frac{1}{HW} \sum_{i,j} F_{i,j,c}$ est le Global Average Pooling, et $\mathbf{W}_1 \in \mathbb{R}^{C/r \times C}$, $\mathbf{W}_2 \in \mathbb{R}^{C \times C/r}$ avec r le ratio de réduction.

3.5 Module d'attention spatiale (CBAM)

Le projet ajoute un module CBAM (Convolutional Block Attention Module) entre le backbone et le classifieur. Il combine attention par canal et attention spatiale.

3.5.1 Attention par canal

$$\mathbf{M}_c(\mathbf{F}) = \sigma\Big(\mathrm{MLP}\big(\mathrm{AvgPool}(\mathbf{F})\big) + \mathrm{MLP}\big(\mathrm{MaxPool}(\mathbf{F})\big)\Big) \tag{14}$$

Le MLP partagé a une couche cachée de dimension C/r :

$$\mathrm{MLP}(\mathbf{z}) = \mathbf{W}_2 \, \mathrm{ReLU}(\mathbf{W}_1 \, \mathbf{z}) \tag{15}$$

3.5.2 Attention spatiale

Après recalibrage des canaux ($\mathbf{F}' = \mathbf{M}_c \odot \mathbf{F}$) :

$$\mathbf{M}_s(\mathbf{F}') = \sigma\Big(f^{7\times7}\big([\mathrm{AvgPool}_c(\mathbf{F}');\ \mathrm{MaxPool}_c(\mathbf{F}')]\big)\Big) \tag{16}$$

où $\mathrm{AvgPool}_c$ et $\mathrm{MaxPool}_c$ agrègent le long de la dimension des canaux, et $f^{7\times7}$ est une convolution 7×7.

La sortie finale est :

$$\boxed{\widetilde{\mathbf{F}} = \mathbf{M}_s(\mathbf{F}') \odot \mathbf{F}'} \tag{17}$$

Pourquoi l'attention spatiale en imagerie médicale ?

Les lésions de la rétinopathie diabétique (micro-anévrismes, hémorragies, exsudats) sont des structures *localisées*. Le module d'attention spatiale permet au réseau de se focaliser sur ces régions discriminantes plutôt que de traiter uniformément toute l'image.

3.6 Transfer Learning et Fine-Tuning

Principe du Transfer Learning

Un modèle pré-entraîné sur ImageNet (14M images, 21k classes) a appris des représentations visuelles génériques (contours, textures, formes). Ces représentations sont transférables à de nouvelles tâches avec peu de données.

Phase 1 — Warmup. Le backbone est gelé ($\nabla_{\theta_{\text{backbone}}} = \mathbf{0}$). Seul le classifieur est entraîné avec un learning rate élevé ($\eta = 10^{-3}$).

Phase 2 — Fine-tuning. Les derniers blocs du backbone sont dégelés. Un learning rate réduit ($\eta = 10^{-4}$) est utilisé pour ajuster finement les features sans détruire les représentations apprises.

4 Fonctions de perte

4.1 Cross-Entropy catégorielle

Pour K classes, la Cross-Entropy entre la distribution vraie $\mathbf{y}$ (one-hot) et la distribution prédite $\hat{\mathbf{y}}$ est :

$$\mathcal{L}_{\text{CE}}(\mathbf{y}, \hat{\mathbf{y}}) = -\sum_{k=1}^{K} y_k \log(\hat{y}_k) \tag{18}$$

Pour un one-hot $\mathbf{y}$ avec $y_c = 1$ (classe correcte), cela se simplifie en :

$$\mathcal{L}_{\text{CE}} = -\log(\hat{y}_c) = -\log(p_t) \tag{19}$$

où p_t est la probabilité attribuée à la classe correcte.

4.2 Focal Loss

Problème du déséquilibre

Dans le dataset de rétinopathie, le grade 0 (No DR) représente ∼73% des images. La CE classique est dominée par ces exemples faciles et fréquents, au détriment des cas rares mais cliniquement critiques (grades 3–4).

La Focal Loss (Lin et al., 2017) module la CE par un facteur $(1 - p_t)^\gamma$:

$$\boxed{\mathcal{L}_{\text{FL}}(p_t) = -\alpha_t \cdot (1 - p_t)^\gamma \cdot \log(p_t)} \tag{20}$$

Rôle du paramètre γ (focusing parameter).

Cas	p_t	$(1-p_t)^\gamma$ **avec** $\gamma = 2$
Bien classé (facile)	0.95	$(0.05)^2 = 0.0025$
Modérément classé	0.70	$(0.30)^2 = 0.09$
Mal classé (difficile)	0.20	$(0.80)^2 = 0.64$

Le facteur focal réduit la contribution des exemples faciles de $\times 400$ par rapport aux exemples difficiles. Le modèle est ainsi forcé de se concentrer sur les cas ambigus.

Pondération par classe α_t. En plus de la modulation focale, un poids par classe est appliqué :

$$\alpha_c = \frac{N}{K \cdot N_c} \tag{21}$$

où N est le nombre total d'échantillons, K le nombre de classes et N_c le nombre d'échantillons de la classe c. Les classes rares reçoivent un poids plus élevé.

4.3 Label Smoothing

Au lieu d'utiliser des labels one-hot durs ($y_c \in \{0, 1\}$), le label smoothing remplace :

$$y'_k = \begin{cases} 1 - \varepsilon + \frac{\varepsilon}{K} & \text{si } k = c \text{ (classe correcte)} \\ \frac{\varepsilon}{K} & \text{sinon} \end{cases} \tag{22}$$

avec $\varepsilon = 0.05$. Cela empêche le modèle d'être trop confiant et agit comme un régularisateur.

4.4 Quadratic Weighted Kappa (QWK) Loss

Le QWK est la métrique standard pour la classification ordinale de la rétinopathie :

$$\kappa = 1 - \frac{\sum_{i,j} w_{ij}\, O_{ij}}{\sum_{i,j} w_{ij}\, E_{ij}} \tag{23}$$

où :

- O_{ij} : matrice de confusion normalisée (observée).
- $E_{ij} = \frac{h_i^{\text{true}} \cdot h_j^{\text{pred}}}{N^2}$: matrice attendue sous indépendance.
- $w_{ij} = \frac{(i-j)^2}{(K-1)^2}$: matrice de pénalité quadratique.

La pénalité quadratique signifie qu'une erreur $0 \to 4$ est pénalisée 16 fois plus qu'une erreur $0 \to 1$, ce qui reflète l'importance clinique.

Pour rendre le QWK différentiable, on utilise une matrice de confusion "soft" :

$$O_{ij}^{\text{soft}} = \frac{1}{N} \sum_{n=1}^{N} y_{n,i} \cdot \hat{y}_{n,j} \tag{24}$$

et la loss à minimiser est $\mathcal{L}_{\text{QWK}} = 1 - \kappa$.

5 Augmentation des données

5.1 Augmentations géométriques

Les images de fond d'œil sont *invariantes par rotation* (pas d'orientation anatomique fixe) et par réflexion, ce qui autorise des transformations géométriques agressives :

- **Rotations** de 0, 90, 180, 270 (sans perte d'information).
- **Flip horizontal** et **vertical**.

Cela multiplie par 8 la diversité effective du dataset.

5.2 MixUp

MixUp (Zhang et al., 2018) crée des exemples virtuels par combinaison convexe :

$$\tilde{\mathbf{x}} = \lambda\, \mathbf{x}_i + (1-\lambda)\, \mathbf{x}_j \tag{25}$$

$$\tilde{\mathbf{y}} = \lambda\, \mathbf{y}_i + (1-\lambda)\, \mathbf{y}_j \tag{26}$$

où $\lambda \sim \text{Beta}(\alpha, \alpha)$ avec $\alpha = 0.2$. La distribution Beta avec un petit α produit des λ proches de 0 ou 1, de sorte que les mélanges restent proches d'un des deux exemples originaux.

Effet sur l'espace de décision

MixUp encourage le modèle à apprendre des frontières de décision *linéaires* entre les classes, ce qui :

- Réduit l'overfitting (régularisation implicite).
- Améliore la calibration (les probabilités de sortie sont plus fiables).
- Renforce la robustesse aux perturbations adversariales.

5.3 CutMix

CutMix (Yun et al., 2019) remplace un patch rectangulaire d'une image par un patch correspondant d'une autre image :

$$\tilde{\mathbf{x}} = \mathbf{M} \odot \mathbf{x}_i + (\mathbf{1} - \mathbf{M}) \odot \mathbf{x}_j \tag{27}$$

$$\tilde{\mathbf{y}} = \lambda\, \mathbf{y}_i + (1-\lambda)\, \mathbf{y}_j \tag{28}$$

où $\mathbf{M} \in \{0,1\}^{H \times W}$ est un masque binaire rectangulaire et $\lambda = 1 - \frac{r_w \cdot r_h}{W \cdot H}$ est la proportion de l'image originale conservée.

La taille du patch est tirée aléatoirement :

$$r_w = W\sqrt{1-\lambda'}, \quad r_h = H\sqrt{1-\lambda'}, \quad \lambda' \sim \text{Uniform}(0,1) \tag{29}$$

6 Optimisation de l'entraînement

6.1 Optimiseur AdamW

AdamW découple la régularisation L2 (weight decay) de l'estimation des moments adaptatifs :

$$\mathbf{m}_t = \beta_1 \, \mathbf{m}_{t-1} + (1-\beta_1) \, \mathbf{g}_t \qquad \text{(moment d'ordre 1)} \tag{30}$$

$$\mathbf{v}_t = \beta_2 \, \mathbf{v}_{t-1} + (1-\beta_2) \, \mathbf{g}_t^2 \qquad \text{(moment d'ordre 2)} \tag{31}$$

$$\hat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1-\beta_1^t}, \quad \hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1-\beta_2^t} \qquad \text{(correction du biais)} \tag{32}$$

$$\boldsymbol{\theta}_t = \boldsymbol{\theta}_{t-1} - \eta \left(\frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon} + \lambda_{\text{wd}} \, \boldsymbol{\theta}_{t-1} \right) \qquad \text{(mise à jour)} \tag{33}$$

Le terme $\lambda_{\text{wd}} \, \boldsymbol{\theta}_{t-1}$ est le weight decay *découplé* : il pénalise directement la norme des poids, indépendamment de l'adaptation du learning rate.

6.2 Réduction du learning rate sur plateau

Quand la loss de validation stagne pendant p epochs (patience), le learning rate est réduit :

$$\eta_{t+1} = \eta_t \times f, \quad f = 0.5 \tag{34}$$

avec un plancher $\eta_{\min} = 10^{-7}$.

6.3 Early Stopping

L'entraînement est arrêté si la métrique de validation (QWK) ne s'améliore pas pendant $P = 10$ epochs consécutives, et les poids du meilleur epoch sont restaurés. Cela prévient l'overfitting en identifiant automatiquement le point optimal.

7 Optimisation du modèle à l'inférence

7.1 Pruning structurel

Le pruning élimine les poids de faible magnitude :

$$w'_{ij} = \begin{cases} w_{ij} & \text{si } |w_{ij}| > \tau \\ 0 & \text{sinon} \end{cases} \tag{35}$$

Le seuil τ est ajusté progressivement via un polynôme pour atteindre le taux de sparsité cible s_f :

$$s_t = s_f + (s_i - s_f) \left(1 - \frac{t - t_0}{t_1 - t_0} \right)^3 \tag{36}$$

avec $s_i = 0$ (sparsité initiale) et $s_f = 0.5$ (50% de poids mis à zéro).

7.2 Quantification

La quantification réduit la précision des poids de float32 (32 bits) à int8 (8 bits) :

$$q = \text{round}\left(\frac{w - z}{s}\right), \quad s = \frac{\max(w) - \min(w)}{2^b - 1} \tag{37}$$

où s est le facteur d'échelle, z le zero-point et $b = 8$ le nombre de bits. Cela réduit la taille du modèle de $\times 4$ et accélère l'inférence sur CPU.

7.3 Test-Time Augmentation (TTA)

Le TTA applique T transformations à chaque image d'entrée et agrège les prédictions :

$$\boxed{\hat{\mathbf{y}}_{\text{TTA}} = \frac{1}{T}\sum_{t=1}^{T} f_\theta(\mathcal{A}_t(\mathbf{x}))} \tag{38}$$

avec $\mathcal{A}_t \in \{\text{id}, R_{90}, R_{180}, R_{270}, H, V, HR_{90}, VR_{90}\}$ (8 transformations).

Estimation de l'incertitude. L'écart-type entre les prédictions TTA fournit une mesure d'incertitude :

$$u_k = \sqrt{\frac{1}{T}\sum_{t=1}^{T}\left(\hat{y}_k^{(t)} - \bar{y}_k\right)^2} \tag{39}$$

Une incertitude élevée signale un cas ambigu nécessitant une attention clinique particulière.

8 Évaluation et métriques

8.1 Matrice de confusion

Pour K classes, la matrice de confusion $\mathbf{C} \in \mathbb{N}^{K \times K}$ est définie par :

$$C_{ij} = \#\{n : y_n = i \text{ et } \hat{y}_n = j\} \tag{40}$$

La diagonale contient les classifications correctes.

8.2 Métriques par classe

$$\text{Précision}_k = \frac{C_{kk}}{\sum_i C_{ik}} \qquad \text{(parmi les prédits } k\text{, combien sont vrais } k\,?) \tag{41}$$

$$\text{Rappel}_k = \frac{C_{kk}}{\sum_j C_{kj}} \qquad \text{(parmi les vrais } k\text{, combien sont détectés}\,?) \tag{42}$$

$$F_1^{(k)} = 2 \cdot \frac{\text{Précision}_k \cdot \text{Rappel}_k}{\text{Précision}_k + \text{Rappel}_k} \tag{43}$$

8.3 AUC-ROC (Area Under the ROC Curve)

Pour chaque classe k (one-vs-rest), la courbe ROC trace le taux de vrais positifs en fonction du taux de faux positifs en faisant varier le seuil de décision τ :

$$\mathrm{TPR}(\tau) = \frac{\mathrm{TP}(\tau)}{\mathrm{TP}(\tau) + \mathrm{FN}(\tau)} \tag{44}$$

$$\mathrm{FPR}(\tau) = \frac{\mathrm{FP}(\tau)}{\mathrm{FP}(\tau) + \mathrm{TN}(\tau)} \tag{45}$$

L'AUC est l'aire sous cette courbe :

$$\mathrm{AUC}_k = \int_0^1 \mathrm{TPR}_k\big(\mathrm{FPR}_k^{-1}(x)\big)\, dx \tag{46}$$

Un AUC de 1.0 indique un classifieur parfait, 0.5 un classifieur aléatoire. L'AUC macro est la moyenne non pondérée des AUC par classe.

8.4 Expected Calibration Error (ECE)

L'ECE mesure l'écart entre la confiance du modèle et sa précision réelle :

$$\mathrm{ECE} = \sum_{b=1}^{B} \frac{|S_b|}{N} \left|\mathrm{acc}(S_b) - \mathrm{conf}(S_b)\right| \tag{47}$$

où S_b est l'ensemble des échantillons dont la confiance tombe dans le bin b, et :

$$\mathrm{acc}(S_b) = \frac{1}{|S_b|} \sum_{i \in S_b} \mathbf{1}[\hat{y}_i = y_i], \quad \mathrm{conf}(S_b) = \frac{1}{|S_b|} \sum_{i \in S_b} \max_k \hat{y}_{i,k} \tag{48}$$

Un modèle parfaitement calibré a $\mathrm{ECE} = 0$.

9 Explicabilité : Grad-CAM

Gradient-weighted Class Activation Mapping

Grad-CAM produit une carte de chaleur montrant quelles régions spatiales de l'image contribuent le plus à la décision du modèle pour une classe donnée.

9.1 Formulation

Soit A_{ij}^k les feature maps de la dernière couche convolutive (canal k, position spatiale (i, j)) et y^c le score de la classe cible c *avant* softmax.

Étape 1. Calcul des poids d'importance par canal via les gradients :

$$\alpha_k^c = \underbrace{\frac{1}{Z}\sum_i \sum_j}_{\text{GAP}} \frac{\partial y^c}{\partial A_{ij}^k} \tag{49}$$

Étape 2. Combinaison linéaire pondérée suivie d'une ReLU :

$$\boxed{L_{\text{Grad-CAM}}^c(i,j) = \text{ReLU}\left(\sum_k \alpha_k^c \cdot A_{ij}^k\right)} \tag{50}$$

La ReLU ne conserve que les contributions *positives* (régions qui augmentent le score de la classe c).

Étape 3. Upsampling bilinéaire à la résolution de l'image d'entrée et normalisation dans $[0, 1]$.

Importance clinique

En imagerie médicale, l'explicabilité n'est pas un luxe mais une nécessité. Un radiologue ne peut pas faire confiance à un modèle "boîte noire". Les cartes Grad-CAM permettent de vérifier que le modèle regarde les bonnes zones (micro-anévrismes, hémorragies, néovascularisation) et non des artefacts (bords de l'image, reflets).

10 Récapitulatif des notations

Symbole	Signification
$\mathbf{I} \in \mathbb{R}^{H \times W \times 3}$	Image d'entrée (hauteur H, largeur W, 3 canaux RGB)
G_σ	Noyau gaussien de variance σ^2
K	Nombre de classes ($K = 5$: grades 0 à 4)
p_t	Probabilité assignée à la classe correcte
γ	Paramètre de focalisation de la Focal Loss
α_t	Poids de la classe t dans la Focal Loss
κ	Quadratic Weighted Kappa
λ	Coefficient de mélange MixUp / CutMix
η	Learning rate
$\boldsymbol{\theta}$	Paramètres du modèle
$\mathbf{m}_t, \mathbf{v}_t$	Moments d'ordre 1 et 2 (AdamW)
s, z	Facteur d'échelle et zero-point (quantification)
A_{ij}^k	Feature map du canal k à la position (i, j)
α_k^c	Poids Grad-CAM du canal k pour la classe c

Fin du document — Fondements mathématiques et algorithmiques